

Project Plastic Waste
Eksamensprojekt i DDU (Teknikfag A)
17. maj 2026

Indsæt billede af produkt eller andet relevant

Gruppemedlemmer

Jeppe Bøgeskov — jepp9920@zbc.dk
Alexander Schade — alex245h@zbc.dk

Vejleder(e)

Michael Witus Schierup — miws@zbc.dk

Forord

nævne Jacs' force majeure, sidenhen programmering vs. artist-aspectet, afrundingsvist ,selvsagt, takke Miws for et godt og lærerigt år og desuden

Indhold

1	Indledning	2
2	Produktprincip	3
3	Projektoplæg	4
4	Problemanalyse	5
5	Software og materialelister	7
6	Projektafgrænsning	10
6.0.1	Relevante oplægkrav	10
7	Planlægning	11
7.1	L ^A T _E X ToDo-notes	11
8	Metoder og redskaber	13
8.1	MoSCoW-metoden	13
8.2	Brainstorming	13
9	Produktudvikling	14
9.1	Flow	14
9.2	Movement-mechanic - white-boxing	14
9.3	Valg af stilart samt inputmetode	15
9.4	Pixel-art	16
9.5	SFX og musik	18
10	Produktevaluering	20
11	Markedsføring og udgivelse	22
11.1	Prisovervejelser	22
11.2	Platformovervejelser	22
11.3	Udgivningsstadie	23
	Bibliografi	28
12	Bilag	1
.1	Appendiks A “brainstorm”	1
.2	Appendiks B “afleverede projektbeskrivelse”	1
.3	Appendiks C “opløsningsovervejelser”	1
.4	Appendiks D “stand-up-referater”	1
.5	Appendiks E “Sprites”	1
.6	Appendiks F “Moodboard”	1

1 Indledning

Skriv om opbygningen af rapporten, hvordan man navigerer den bedst, hvorfor vi har gjort de valg, vi har gjort i opstillingen. Beskriv desuden, hvordan de forskellige kapitler hænger sammen, hvordan produktudviklingssektionen er kronologisk, og hvordan produktevaluering er fjernet fra bare debugging, men i stedet er brugt til at træffe valg og oplyse (gøre opmærksom på ikke-identificerede problemstillinger)

Kapitel: Problembeskrivelse

Kapitel: Resume

2 Produktprincip

Tilføj elevatorpitch

3 Projektoplæg

Teknikfag A – Digitalt Design og Udvikling
Slagelse HTX - 2026

Projektoplæg 2

Tema

Opsamling af plastik

Formål

Udvikling af et læringsspil der omhandler indsamling af plastik og hvorfor det er et problem, hvis det ikke gøres, eks.: Resource mangel eller plastik øer i Stillehavet

Beskrivelse

Spillet skal skabe awareness om plastik oprydning og hvad det vil sige at være ansvarlig for hvad man efterlader, særligt mht. plastik. (det kunne også at stille spørgsmålstejn til hvorfor der ikke er en eneste plastik skraldespand på skolen – måske er man på quest efter plastik?).

Særlige krav (husk i bliver målt på disse til eksamen)

1. Brugeren skal igennem spillet lære at plastik skal indsamles og afleveres korrekt.
2. Spillet kan anvende simulering, eks. Af et økosystem der bliver fyldt med plastik.
3. Spillet skal have en modstander med grundlæggende egenskaber (eks. Patrol script)
4. Adskillige assets skal være lavet fra bunden og have udførlig begrundet sammenhæng med en valgt målgruppe.
5. Spillet skal gennemføre minimum 5 runder brugertests, der anvendes til at forbedre spillet.
6. Spillet skal som minimum have en funktionel og en æstetisk prototype.
7. Produktet skal deles eller afleveres i en interaktiv udgave.
8. Produktet skal optages som en "playthrough" film via screengrab eller med mobil.

4 Problemanalyse

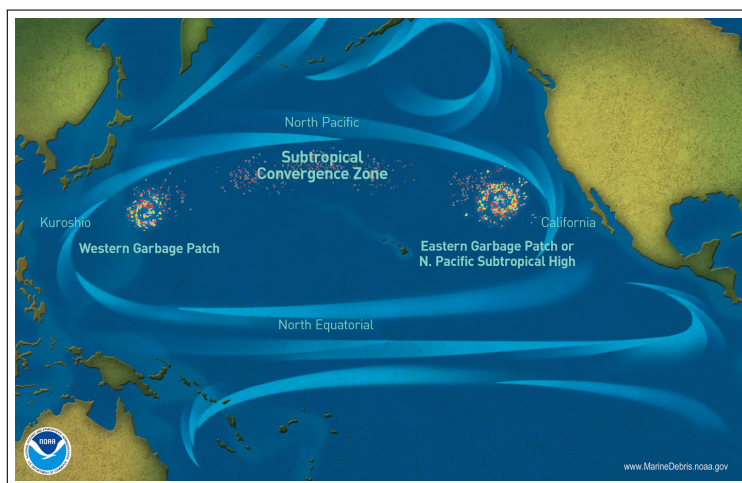
Indsæt målgruppevurdering

Indsæt research om klimafortaleres budskab

Indsæt research om havets plastik-forurening

Plastik forurening udgør i havene et stadigt voksende problem. Store mængder af plastikaffald bliver udledt i havet hvert år og samles i havstrømmene, hvor de danner »garbage patches«¹.

Den mest kendte og den absolut største af disse ophobninger er *The Great Pacific Garbage Patch*, som befinder sig i det nordlige Stillehav. Forskning viser, at *The Great Garbage Patch* indeholder omkring 1,8 billioner stykker plastik (dvs. 1.800 milliarder) spredt over et område på omkring 1,6 millioner kvadratkilometer². Det er svært at forestille sig, hvor stort det er. Men til sammenligning er hele Danmarks areal er lidt over 43.000 kvadratkilometer, så det svarer til at have mere end 37 Danmark i størrelse fyldt med plastikaffald.



Figur 1: The Great Pacific Garbage Patch Cleanup, 2026

Plastiofhobningerne skyldes, at når plastikaffald ender i havet, bliver det transporteret af havstrømmene og samles i specifikke områder i havet, hvor strømmene mødes og skaber cirkulære bevægelser. Disse strømme kan ses på figur ?? som viser *The North Pacific Gyre*.

Plastik i havet udgør en meget alvorlig trussel mod havets økosystemer og dyreliv. Mange dyr i havet forveksler plastik med mad og spiser det fejlagtigt, hvilket fører til hæftige skader på deres fordøjelsessystemer og i værste fald død. Plastik nedbrydes ikke let i naturen og nedbrydes i stedet til mindre og mindre stykker, der kan være endnu mere skadelige for dyrelivet, og det gør det også nært umuligt at fjerne fra havet igen.

¹National Geographic Society (apr. 2025). *Great Pacific Garbage Patch*. Dateret 20. april 2026. URL: <https://education.nationalgeographic.org/resource/great-pacific-garbage-patch/>.

²The Ocean Cleanup (2026). *The Great Pacific Garbage Patch*. Dateret 23. april 2026. URL: <https://www.theoceancleanup.com/great-pacific-garbage-patch>.

Den palstik som bliver nedbrudt til mindre dele, og ender i fiskene, har en direkte effekt på mennesker, da vi spiser fiskene får vi det mikroplastik ind i vores krop.

Indsæt overvejelser om brugeres skærmopløsning jf. steam surveys

Steam er en digital platform som distribuerer spil til millioner af brugere verden over.

Steam udgiver, hver måned en undersøgelse som deres brugere frivilligt kan deltage i. Undersøgelsen omhandler hvilken hardware og software deres brugere har. Disse tal er utroligt vigtige for os, da det giver os en indikation af, hvilken hardware vores spil skal kunne køre på.³

Vi notere os, at 51,93% af Steam brugere har en skærmopløsning på 1920x1080 og udgør derfor normen for skærmopløsning

³Valve Steam (mar. 2026). *Steam User Survey 2026*. URL: <https://store.steampowered.com/hwsurvey/Steam-Hardware-Software-Survey-Welcome-to-Steam>.

5 Software og materialelister

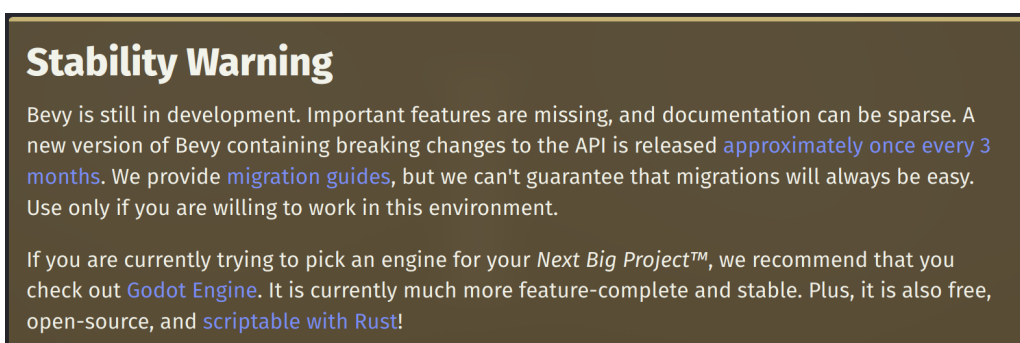
Bevy *Bevy* er en spilmotor, bygget i Rust, se sektion 5, og distribueres som en Rust-package.

```
use bevy::prelude::*;
```

Listing 1: Her ses Bevy-biblioteket blive importeret, fra kodens `main.rs`-fil.

En af fordelene ved Bevy, udover at det er en moderne spilmotor, er, at Bevy er fuldstændig datadrevet, hvilket også er meget effektivt og simpelt. Spilmotoren er i skrivende stund uden en game editor, og derfor skal alting kodes direkte. Det er imidlertid ikke en ulempe, da begge gruppemedlemmer, herunder undertegnede, i forvejen har programmeringskompetencer. Såfremt udviklingsteamet var større, og især, hvis der er ikke-programmører på teamet, vil det være et problem. En game editor er dog i Bevys pipeline.

Desuden er Bevy en relativt ny spilmotor, der udkom i 2020⁴, hvilket nødvendigvis indebærer, at spilmotoren ikke er ligeså moden som sine alternativer. På Bevys hjemmeside, beskriver udvikleren selv, at de fraråder større eller kommercielle projekter at anvende Bevy til fordel for andre motorer, da spilmotorens syntaks ivrigt ændres (de har dog manualer til at oversætte forældet kode, så man kan migrere sit projekt til en ny version), hvorfor ens spil kan gå i stykker, hvis man migrerer uden at tage de nødvendige forbehold, se Figur 2.



Figur 2: Viser Bevys “stability warning” under deres “quick start”-side. Siden kan findes på <https://bevy.org/learn/quick-start/introduction/>.

Selve Bevys projekter indeholder en MIT-licens og en Apache 2.0-licens. Begge licenser tillader, at man gør sin software privatejet. Det er vigtigt, idet nogle open source-software har såkaldte copyleft-licenser. Det er kutyme, at kommerciel software er privatejet, da man i så fald kan beskytte koden, der er grundlag for ens unikke spil, og derfor indtjeningsgrundlag.

ECS-paradigmet Bevy er som tidligere beskrevet datadrevet. De tre primære byggede dele er henholdsvis entiteter, kommandoer og systemer. Entiteter er det, som eksisterer i ens program. De udgøres af komponenter, der kan indeholde en given data eller information. Kommandoer er kontaktfladen til entiteterne, med kommandoer kan vi oprette en

⁴Carter Anderson (2022). *Bevy's First Birthday*. URL: <https://bevy.org/news/bevys-first-birthday/>.

ny entitet eller fjerne entiteter. Systemer kan kalde kommandoer, kan lede efter bestemte entiteter og foretage ændringer på bestemte komponenter. Systemerne kan eksekveres betinget. Eksempelvis kan vi have et system, der kører for hver ny frame og opfanger input, og hvis dette input matcher et ønsket input, gør systemet noget, eksempelvis modificere spillerens bevægelseskompont, så denne rykkes i en bestemt retning.

Komponenterne er Rust-structs, ikke-indbyggede datatyper. Det kan være simple tuple-structs, hvor at structen bare er en container for en anden datatype, måske en mere primitiv datatype. På listing 2 ses eksempelvis, vores **Velocity**-komponent, således at denne kan indgå som del af en entitet, vi således kan anvende et system på.

```
#[derive(Component)]  
pub struct Velocity(pub Vec2);
```

Listing 2: Viser **Velocity**-komponentet bliver oprettet.

I Bevy er der også mange features, eksempelvis resurser. Resurser er universel data, der kan tilgås for sig selv. I vores projekt har vi en resurse, der holder styr på mængden af bølger, der er tilbage, eller mængden af point, spilleren har. Disse kan også hentes af systemer. Indbyggede resurser er eksempelvis også tid; der bruges, når vi skal bruge delta-tid i systemer.

Alternative spilmotorer, der havde været værd af betragte i kommercielle sammenhæng havde været Unity eller Unreal Engine. Godot har vi ikke nogle erfaringer med, så det havde været en byrde.

Rust Rust er programmeringssproget, som projektet her er skrevet i. Vi har valgt dette programmeringssprog, da det er et hurtigt, low-level sprog, vi har erfaring med fra programmering. Rust har desuden en række andre fordele, eksempelvis sin borrow checker, der sikrer data-integritet og data-sikkerhed samt en compiler, der håndhæver god kode-skik. cargo-suiten gør det nemt at kompilere binærer, og Rust har også et rigt økosystemer med pakker, der kunne være relevante for vores projekt, eksempelvis **serde** og **dirs**. Alternativt kunne vi have brugt Python, da vi har erfaring hermed, men det er mere et script-sprog og fortolker koden i realtid, hvilket ville gøre det svære i distributionsfasen, men er fuldstændig ladsiggørligt.

Havde vi anvendt Unity, havde vi skrevet i C#, men det har vi ikke erfaring med, udover tidligere projekter i Unity.

Photopea Photopea er en webapp, som forsøger at replikere det meget succesfulde og kendte Photoshop. Vi valgte at benytte Photopea, da vi tidligere har erfaring med Photoshop, men det er desværre ikke kompatibelt med vores nuværende styresystem. Photopea er som førnævnt en webapp, hvilket også kommer med signe begrænsninger, herunder performance-begrænsninger.

Vi har benyttet Photopea til at producere alle Sprites til spillet.

Doom Emacs Til primært at skrive koden til programmet er Doom Emacs anvendt, en modifikation af Emacs, som er et rigt tekstbehandlingsprogram med så mange pakker, udvidelser, at der eksisterer en joke om Emacs, at det er “a great operating system,

lacking only a decent editor”. Doom Emacs har en masse præ-definerede genvejstaster, præ-definerede pakker og kode-highlighting, der muliggør et godt workflow. Alternativt kunne vi have brugt Microsoft Visual Studio Code, der er industrinorm. En af de helt store fordele ved VsCode er den tætte indbyggede AI-integration; man kan formentlig opnå noget lignende i Emacs.

GitHub GitHub er en gratis webbaseret platform, som kan gemme Git-depoter. Git er et versionshåndteringsværktøj, som er meget udbredt i programmeringsverdenen. Med Git kan vi gemme forskellige versioner af vores kode, følge ændringer over tid og samarbejde om det samme projekt uden at overskrive hinandens arbejde. Dette gør det lettere at udvikle større programmer (som dette), da man altid kan gå tilbage til tidligere versioner, hvis der opstår fejl. GitHub fungerer samtidig som en online backup og giver mulighed for at dele projekter med andre udviklere samt udgive den endelige version af spillet så det let kan nedhentes.

GitHub tilbyder også andre værktøjer end Git. Vi har også benyttet GitHubs SCRUM-funktion samt udarbejdet et Roadmap.

L^AT_EX L^AT_EX er et markupsprog til tekstformatering, hvilket indebærer at man skrive en separat kodefil, som sidenhen kan kompiles til eksempelvis en PDF-fil, der kan distribueres. Dette er i stor modstrid til normal Word processing, hvor man skriver direkte i Rich Text. Det indebærer, at man selv manuelt skal ændre på størrelsen af tekst, dens udseende, mens man i L^AT_EX bare angiver sin teksts struktur og sidenhen kan ændre reglerne for, hvordan det skal kompilles. Det gør, at ens resultat bliver ordentligt og strømlinet.

Det er klart, at der er en afvejning her. Eksempelvis kan du risikere, at dokumentet ikke kompiles, hvis du har lavet en kodefejl; man risikerer debugging. Men i takt med, at ens opgave vokser, bliver det relativt nemmere at administrere i latex, da al nummerering automatisk genereres, bibliografi håndteres automatisk osv. Alle ting som vil kræve manuel formatering med en Word Processor. Desuden er der selvfølgelig mange andre fordele som resultat af L^AT_EX-økosystemet. Matematikbehandlingen alene er også en grund til, at mange foretrækker L^AT_EX fremfor alternative løsninger til fremstilling af rapporter. Af de grunde har vi også valgt at bruge det.

6 Projektafgrænsning

Til prioritering af features i spillet, anvendes MoSCoW-metoden, se under-sektion 8.1. Prioriteringerne kan ses i Tabel 1.

Uddyb valg af prioriteringer

Must have	• Alle relevante oplægkrav* • PlayerAvatar • Skyde- og bevægelsessystem
Should have	• Controller-keyboard-support
Could have	• Local Co-Op
Won't have	• Multiplayer / Networking

Tabel 1: I figuren ses prioriteringerne fordelt baseret på vigtighedsgrad jf. under-sektion 8.1.

*Alle relevante eksamenskrav kan ses i under-under-sektion 6.0.1

6.0.1 Relevante oplægkrav

De relevante krav fra oplægget, der er obligatoriske er her opstillet og oversat. Jf. oplægget, se sektion 3.

1. Brugeren skal undervises i korrekt håndtering af plastikaffald.
2. I spillet anvendes økologisk simulering.
3. Spillet skal indeholde en fjendtlig NPC med patrolscript.
4. Adskillige assets skal være lavet fra bunden og udførlig begrundet sammenhæng den valgte målgruppe
 - En målgruppe skal vælges (Under alle omstændigheder).
5. Spillet skal itereres ad fem gange med baggrund i test af spil.
6. Det endelige produkt skal bestå af henholdsvis en æstetisk og funktionel prototype. (**note** en prototype kan være begge)
7. Produktet skal udgives interaktivt. (**note** kompilere binærer)
8. En filmet gennemspilning (playthrough) af spillet skal vedhæftes projektet.

7 Planlægning

I projektets spæde start, optegnede vi med GitHubs projekt funktion, en Sprint-plan for hele projektets periode. En Sprint, betegner en arbejdsuges/skoleuge som i vores tilfælde er mandag til fredag. Planen så således ud:

Periode	Opgave
Uge 11	Projektstart
Uge 12	Controller movement
Uge 13	Opløsning og skalering (bar)
Uge 13	Pixel art
Uge 15	Monsterlogik (patrol script)
Uge 15	Skydesystem
Uge 16	Shopsystem og meta-progression
Uge 17	Intensiv rapportskrivning
Uge 19	Bufferperiode
Deadline 18/05	Aflevering

Tabel 2: Sprint Plan – DDU Teknikfagsprojekt

Denne plan afspejler selvfølgelig ikke hvordan projektet forløb i virkeligheden, da ting altid tager længere tid end forventet. Vi har løbende måtte fortage ændringer i den plan for at tage hånd om dele som enten har været foran eller bagud for planen. Vi har måtte håndtere en hel del uforudsete udfordringer som sygdom, som har gjort at vi måtte ændre i planen. Den omplanlægning har ikke været af aller højeste kvalitet, men selvom det har projektet aligevel lykkedes. Det skyldes at gruppens størrelse ikke var større end den er og vi har derfor kunne fortage beslutninger ad hoc som man ellers ikke normalt i større spiludviklingsprojekter ville gøre.

Angiv standup-referater og link til SCRUM

Nævn Git

Visualiser forgrening af GitHub-projekt

7.1 L^AT_EX ToDo-notes

Selve rapporten her er skrevet i L^AT_EX-kode, læs om L^AT_EX i sektion 5, og sidenhen kompileret via `pdflatex`. En særlig L^AT_EX-package, er `todonotes`, der gør det muligt at oprette to-do-noter, der så inkluderes i den kompilerede pdf. Det har muliggjort planlægning af strukturen i rapporten direkte i rapporten, og således har vi ikke behøvet et separat system til at holde styr på rapportens indhold. Det betyder også, at man bliver konfronteret med noterne undervejs, og man bliver således nødt til at forholde sig til dem. To-do-noterne

er også blevet brugt som et rudimentært skelet, og som færdighedsmarkør—hvor at en ToDo-note viser, at et kapitel ikke er færdigskrevet. Af Figur 3 fremgår, hvordan disse todonotes ser ud, når L^AT_EX-dokumentet er kompileret.



Figur 3: Viser en to-do-note fra pakken `todonotes`. Her oprettet via `...` osv. Bemærk, at den er oprettet med `inline`, da to-do-noten ellers placeres i dokumentets margen, hvilket kan se mærkeligt ud, især når indholdet i todo-noten er tilpas stor.

8 Metoder og redskaber

8.1 MoSCoW-metoden

Skriv om MoSCoW-metoden

Angiv brugte redskaber og metoder

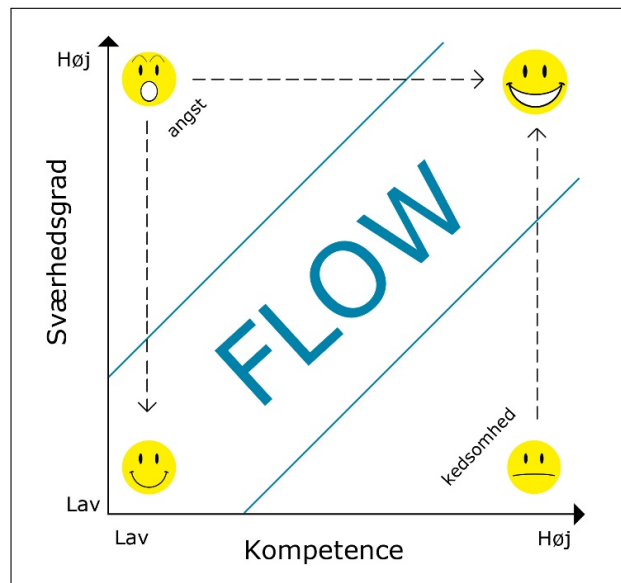
8.2 Brainstorming

Skriv om brainstorming

9 Produktudvikling

9.1 Flow

Før vi begynder med den konkrete udvikling af spillet, er det vigtigt, helt strukturelt, at vi opbygger spillet med flow-teorien⁵ i tankerne. Flow-teorien beskriver, hvordan en spiller bedst bliver underholdt når han spiller et computerspil. Hvis ens computerspil for hurtigt viser en meget høj sværhedsgrad, så giver det spilleren en angstlig følelse, hvilke virker negativt på hans lyst til at fortsætte med at spille spillet, og det er ikke hensigtsmæssigt. Hvis spillet derimod er af en lav sværhedsgrad og spillerens kompetenceniveau er for stort, vil spilleren kede sig og afbryde spillet. Hensigten er derfor at skabe et spil som gradvist øger sværhedsgraden sammenløbende med at spillerens kompetencer forøges.



Figur 4: Viser flowstadiet. Kilde: Frans Ørsted Andersen, 2022

Som vi beksriver senere i rapporten, bliver spillet på et tidspunkt udgivet i en Early Access udgave, her vil vi få en større spillerbase og teste flow på. Vi har derfor en masse konstanter i spillet som vi i Early Access vil kunne skrue på for at opnå det optimale flow.

9.2 Movement-mechanic - white-boxing

Det første, der implementeredes, var et rudimentært bevægelsessystem for spillerens avatar. Spilleren kan accelereres i alle hovedretningerne, altså nord, syd, øst og vest. For spilleren holdes der til enhver tid styr på spillerens hastighed $\vec{v}$ og position $\vec{OP}$ i form af vektorer. Accelerationen $\vec{a}$ er ikke bundet til spilleren, men er givet øjeblikkeligt og påvirker altså hastigheden, der igen påvirker positionen.

Når accelerationen er givet, opdateres $\vec{v}$ ved at tillægge ændringen i hastighed $\Delta\vec{v}$, der svarer til den øjeblikkelige multipliceret ændringen i tid, til den gamle hastighed $\vec{v}_0$

$$\vec{v}_1 = \vec{v}_0 + \Delta\vec{v} = \vec{v}_0 + \Delta t \cdot \vec{a} \quad (1)$$

⁵Wikipedia (2026a). *Flow (psychology)*. Wikipedia.com. URL: [https://en.wikipedia.org/wiki/Flow_\(psychology\)](https://en.wikipedia.org/wiki/Flow_(psychology)).

Herefter opdateres placeringen af spilleren $\vec{OP}$ ved at tillægge ændringen i position

$$\vec{OP}_1 = \vec{OP}_0 + \Delta\vec{P} = \vec{OP}_0 + \Delta t \cdot \vec{v} \quad (2)$$

Således er der altså anvendt Newtonisk fysik; det er oplagt, da det er let at have med kræfterne, vi har at gøre med, mens vi har anvendt vektorregning, da vektorer har retning og størrelse, hvilket kræfter også har. Således behøver vi ikke at forholde os til x- og y-koordinaterne direkte. Udover at det er nemmere at kode, bør det også introducere færre fejl, hvor det sidste kommer slutbrugeren til gode.

Selve accelerationen er givet ved, brugerens input (dynamic) til ethvert øjeblik. Følgende keyboardtaster korresponderer til følgende

- **w** → acceleration i nordlige retning
- **a** → acceleration i vestlige retning
- **s** → acceleration i sydlige retning
- **d** → acceleration i østlige retning

Bemærk, at dette er alle hovedretninger. Valget på netop de taster falder på, at det er etablerede konvention. Således bør det gerne være intuitivt for spilleren, og denne bør hurtigt kunne tilegne sig en evne til at bevæge sig, hvilket bør vække tilfredsstillelse hos spilleren.

Til at teste, brugte vi et simpelt mesh i en simpel 2D-verden, og det virkede. Nu gennemgås implementeringsdetaljerne, for relevant Bevy-terminologi, se sektion 5 og sektion 5

Skriv om implementering af Movement-system (logik) (white-boxing)

Skriv om implementering af Skyde-system (logik)

Skriv om spillets stil (kunst)

9.3 Valg af stilart samt inputmetode

Til vores spilprojekt har vi valgt at lave alt i pixel-art-stilen. Pixel-art er en moderne form for digital kunst, hvor man replikere stilen fra ældre computerspil, hvor man dengang havde markante hardwarebegrænsninger som gjorde at grafikken havde meget lavere opløsning end i dag. I moderne tid bruger spiludviklere ofte pixel-art til bevidst at skabe et retro- eller arkadeinspireret udtryk, hvilket passer godt ind i hvad vi ønsker med spillet.

Pixel-art stilen appellerer til vores målgruppe, europæiske gymnasieelever, gennem sit retro- og arkadeinspirerede udtryk.

Vi har ligeledes valgt at udvikle spillet til controller, for at ydeliger appellerer til vores målgruppe. Dette giver en mere analog oplevelse af spillet.

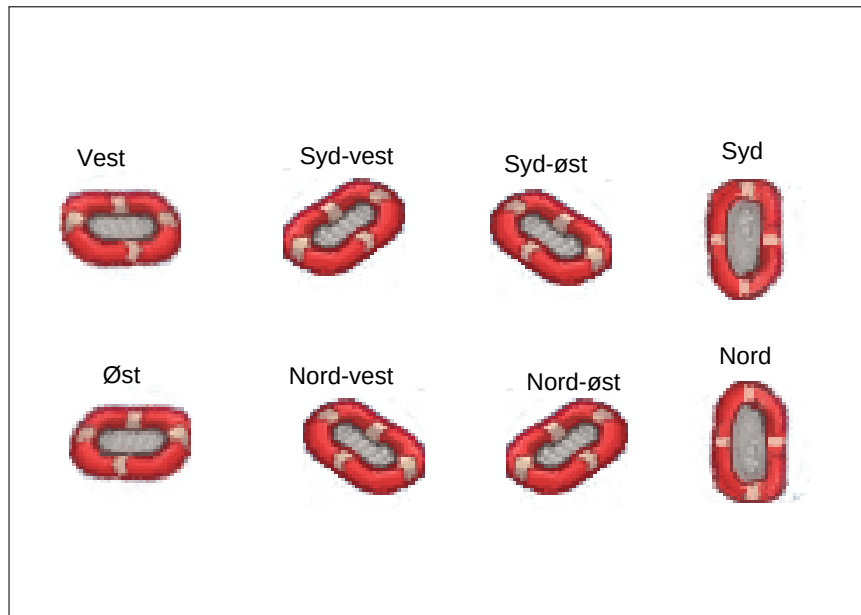
Skriv om udviklingen af pixel-art (kunst)

9.4 Pixel-art

Til udviklingen af spillet har vi måtte producere en række grafiske elementer, som tilsammen skaber spillets visuelle udtryk. Selvom vores målgruppe ikke har et nostalgisk tilhørsforhold, der måske endda kan være kontraproduktivt, til pixel-art og datidens originale hardwarebegrønsninger, men til gengæld har de en romantisk forestilling om arkadekulturen, en anemoia (græsk ánemos ”vind” og -noia ”sind”).⁶

Da spillet er udviklet i pixel-art-stilen, er alle sprites designet med lav opløsning og i et begrænset antal farver for at give spillet et retro- og arkadeinspireret udtryk. De fleste sprites er udviklet i opløsninger mellem (26 px × 16 px) og (80 px × 80 px), hvilket både understøtter pixel-art-stilen og reducerer spillets ressourceforbrug. Dette valg passer både til målgruppen (europæiske gymnasieelever) og til spillets relativt simple gameplay, samtidig med at det gør de forskellige objekter tydelige for spilleren.

Et af de vigtigste grafiske elementer er den båd spilleren sejler rundt i. For at skabe en mere dynamisk oplevelse har vi designet sprites til båden i flere retninger, herunder: *Vest*, *Syd-vest*, *Syd-øst*, *Syd*, *Øst*, *Nord-vest*, *Nord-øst* og *Nord*. Spilleren, som sidder i båden, har ligeledes separate sprites til de forskellige retninger, så karakterens orientering passer til bådens bevægelse. Dette gør animationerne mere troværdige og forbedrer spiloplevelsen. Som vist på figur 5 er båden udviklet med flere retningsbestemte sprites for at skabe mere dynamiske animationer. Båd-sprite er baseret på båden vist i moodboardet (.6). Båd-spritesne er udviklet i opløsningen (35 px × 35 px), hvilket gav tilstrækkelig detaljegrad til at spilleren tydeligt kan aflæse bådens retning uden at bryde med spillets simple visuelle stil.



Figur 5: Sprites til spillerens båd i forskellige retninger.

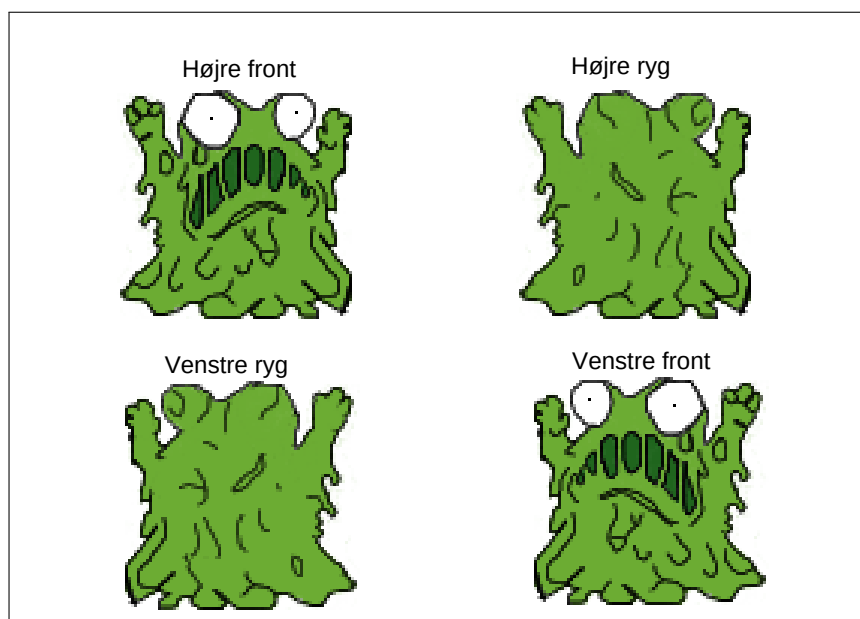
Motoren på båden er udviklet som separate sprites fremfor at være en fast del af båd-sprite. Dette valg blev truffet ud fra både tekniske og visuelle overvejelser. Ved at opdele

⁶Sophie Esther Ramsey (sep. 2023). *Anemoia & Gen Z's '80s dream*. The Varsity. URL: <https://thevarsity.ca/2023/09/10/anemoia-gen-zs-80s-dream/>.

motoren fra resten af båden kan motor-spriten rotere uafhængigt af båd-spritsens retning, hvilket gør det lettere at implementere styring og bevægelse i spillets kode. Samtidig giver det mulighed for mere fleksible animationer og et mere levende visuelt udtryk.

Derudover har vi udviklet forskellige sprites til plastikaffald, som spilleren skal samle op i løbet af spillet. Vi har valgt at fokusere på tre typer affald: en plastikvandflaske, en plastikdunk og en dåse. Disse objekter er valgt, fordi de er genkendelige former for affald, som ofte forbindes med forurening i havet. Ved at gøre affaldet tydeligt og let genkendeligt bliver spillets budskab om plastikforurening mere direkte for spilleren.

Spillet indeholder også en boss-fjende i form af et skraldemonster. Dette monster repræsenterer den ophobning af affald, som findes i havene, eksempelvis i områder som “The Great Pacific Garbage Patch”. Ligesom båden er bossen designet med sprites i flere retninger for at skabe mere flydende bevægelse og gøre mødet med bossen mere dynamisk. Bossens design adskiller sig visuelt fra de normale affaldsobjekter ved at være større og mere detaljeret, så spilleren tydeligt kan identificere den som spillets centrale udfordring. Skraldemonsteret er en videreudvikling af et design vi fandt på Pinterest⁷, Vi har brugt Pinterest-designet, som “skabelon”, og tilpasset det så det passer ind i vores spil samt tilføjet “retninger” så den også kan have ryggen imod skærmen. Se Figur 6, hvor monster-spritesne er vist.



Figur 6: Monster spritesne.

En stor del af spillets assets er udviklet fra bunden for at sikre et ensartet visuelt udtryk og for at kunne tilpasse grafikken specifikt til målgruppen. Pixel-art-stilen blev valgt, da den både er relativt hurtig at producere inden for projektets tidsramme og samtidig appellerer til unge spillere gennem dens retro-inspirerede udtryk. De resterende sprites kan ses på appendiks .5

Trawler

Trawler-assten er blevet lavet med generativ AI (specifikt ChatGPT) Vi har valgt at

⁷<https://uk.pinterest.com/pin/trash-pack-character-trashola-pixel-art--40180621656504620/>

bruge generativ AI til at lave denne asset, da den er ret kompliceret, og vores grafiske og kreative enskaber må siges at være begrænsede. Trawleren skal være inde for $150\text{px} \times 150\text{px}$ og den skal være i profil. Vi har derfor promptet modellen med: *“lav en oliefedt trawler i profil i stilen pixel-art, den skal være øst-gående. $150\text{px} \times 150\text{px}$ ”*. Promptet gav faktisk første gang to meget brugbare assets, disse fortager vi en A/B test af for at finde ud af hvilken er bedst til vores formål. Denne A/B test foretog vi uformelt i person, hvor vi spurgte 10 venner/bekendte hvilken de syntes bedst om til vores skraldeopsamlingsspil. Vi fortalte dem at trawleren skulle være ond og oliefedt. Ud af de 10, mente 7 at trawler A var den bedste til formålet.



Figur 7: Billeder af AI-genereret trawler assets brugt til A/B test

Nu hvor trawleren blev valgt, skal den klargøres til at indsættes i spillet. Det betyder at den skal klippes ud af baggrunden som AI-modellen ikke ville fjerne. Dette gjorde vi ved Photopea, hvor vi benyttede Magic Wand-featureen. Dernæst spejlede vi den om y-aksen for at få begge retninger.

Skriv om implementering af pixel-art (transform-overvejelser)

Skriv om SFX og musik

9.5 SFX og musik

Lyddesign spiller en essentiel rolle i computerspil, da det bidrager til både stemning, feedback og spillerens oplevelse af gameplayet. I vores spil har vi arbejdet med forskellige former for SFX (sound effects) og musik for at understøtte spillets arkadeinspirerede stil og 8-bit æstetik. Den lave bitrate og retroinspirerede lydtil er valgt for at skabe en nostalgisk atmosfære, som passer til spillets visuelle udtryk og hurtige gameplay.

Til udviklingen af alle lyd-elementer har vi anvendt royalty free musik og lydeffekter fra platformene Pixabay⁸ og Upbeat⁹. Disse valg gjorde det muligt at benytte musik og lyde uden ophavsretlige problemer, samtidig med at vi kunne finde lydmateriale, der passede til spillets tema da begge indeholder meget store biblioteker af gode lyde med mere.

⁸<https://pixabay.com>

⁹<https://upbeat.io>

Der er implementeret flere forskellige typer lydeffekter i spillet. Blandt andet anvendes skudlyde inspireret af små pistoler for at give spilleren tydelig feedback under fights. Derudover er der tilføjet lydeffekter til opsamling af skrald, så spilleren modtager øjeblikkelig lyd-respons ved succesfulde handlinger. Andre SFX omfatter lyde fra flydende skrald i havet samt lyd ved spawning af monstre, hvilket bidrager til at gøre spilverdenen mere levende og dynamisk.

Musikken er opdelt i forskellige kategorier afhængigt af spilsituationen. Den almindelige baggrundsmusik, er af tre tracks og er alle rolige og understøtter udforskning af havmiljøet, mens bossmusikken er mere intens og tempofyldt for at skabe spænding under boss fights. Derudover er der anvendt ambience-lyde fra havet, eksempelvis bølger og undervandsstemning, som hjælper med at skabe immersion og understøtter spillets miljøtema omkring havet og plastikforurening.

Ligesom pixel-art-stilen appellerer 8-bit-lydestetikken til målgruppen — europæiske gymnasieelever — gennem *anemoia*. De karakteristiske chiptune-lyde¹⁰ og enkle lydeffekter er ikke noget, målgruppen selv har oplevet i deres opvækst, men de kender dem alligevel som et kulturelt og æstetisk sprog fra moderne spil og internetkulturen. Denne genkendelse uden direkte erindring skaber en særlig tiltrækningskraft, der gør lyddesignet medvirkende til at fastlægge spillet i en æstetik, som målgruppen finder autentisk og engagerende. 8-bit-lyden er således ikke blot en praktisk forenkling — det er et bevidst valg, der møder spilleren i et lyddesign, de intuitivt forbinder med netop den type arkadeinspireret gameplay, som spillet tilbyder.

Skriv om begrænsning af sigtekornet (logik)

Skriv om hit detection (logik)

¹⁰Chiptune er en lydstil opstået fra hardwarebegrænsningerne i tidlige spillekonsollers lydchips, kendetegnet ved enkle, syntetiske bølgeformer.

10 Produktevaluering

klargør forskel på interne test versus eksterne tests (inddrager vi udefrakommende?), SO-overvejesr, herunder reliabilitet og repræsentativitet, valg og ræsonnement, samt at de her præsenteres kronologisk, og den afrundende spillertest (af det endelige produkt)

paper-dummy (ekstern)

leftstick for sejlning eller leftstick for skydning og omvendt, evt. mulighed for at ændre? Tesis er at kejthåndede foretrækker venstre til skydning (større præcisionsbehov end for sejlning) (evt. ekstern)

skal højre venstre på bevægelse stick rotere skib og op og ned accelere og decelere (intern)

evt. post mortem

11 Markedsføring og udgivelse

Beskriv antagelse om at spillets aktiver er proprietære (og ikke open-source, som de faktisk er nu grundet nemhedens skyld)

Prisovervejelser

11.1 Prisovervejelser

Hvis vores spil i fremtiden skulle udgives, er det essentielt at vi overvejer en pris. For at vi kan gøre det, vurderer vi at det er hensigtsmæssigt at vi sammenligner vores spil med andre lignende uafhængigt producerede spil på markedet (specifikt på Steam)

Tabel 3 viser en oversigt over udvalgte sammenlignelige titler og deres nuværende priser.

	Genre/stil	Pris (Steam)
<i>Hotline Miami</i>	Top-down arkadeskydespil, pixel-art	\$9,99
<i>Nuclear Throne</i>	Top-down arkadeskydespil, pixel-art	\$11,99
<i>Undertale</i>	Indie RPG, pixel-art, lille scope	\$9,99
<i>Celeste</i>	Indie platformer, pixel-art	\$19,99

Tabel 3: Sammenlignelige indie-spil og deres priser på Steam (maj 2026).

De spil vi har valgt at indrage til sammenligningen, er alle indie-spil (uafhængigt producerede spil) i pixel-art stilen. De spil vi sammenligner os med ligger alle i prislejet \$9,99–\$19,99.

Da vores spil er et skoleprojekt i af begrænset omfang og med begrænsede midler, ville det derfor være urimeligt at udgive og prisætte et semi-færdigt spil på niveau med spil som er udviklet af et professionelt spiludviklingsstudie over flere år. Så vi vurderer at når spillet udgives i Early Access, skal det derfor være gratis, indtil det udkommer i full release. Full release-versionen bør prisættes i prislejet \$5–\$8

Platformovervejelser

11.2 Platformovervejelser

Da der er utalligt mange forskellige styresystemer, er det meget relevant at kigge på hvilke af disse vi bør udgive spillet til. Da vi har lavet spillet via Bevy, er det rimligt nemt at kompilere spillet til alverdens forskellige systemer.

For at udvælge hvilke styresystemer vi skal udgive spillet til, kan vi igen bruge Steam Hardware & Software Survey¹¹ Her kigger vi specifikt på “OS-version”

¹¹Valve Steam (mar. 2026). *Steam User Survey 2026*. URL: <https://store.steampowered.com/hwsurvey/Steam-Hardware-Software-Survey-Welcome-to-Steam>.

Steam Hardware & Software Survey — April 2026

Top 3 operativsystemversioner per kategori

Kategori	OS-version	Andel	Ændring
Windows	Windows 11 64-bit	67,74 %	+1,14 %
	Windows 10 64-bit	25,63 %	+0,27 %
	Windows 7 64-bit	0,07 %	−0,01 %
macOS	macOS 26.3.1 64-bit	0,49 %	0,00 %
	macOS 26.4.0 64-bit	0,27 %	+0,20 %
	macOS 15.4.1 64-bit	0,20 %	+0,20 %
Linux	Arch Linux 64-bit	0,32 %	−0,02 %
	Linux Mint 22.3 64-bit	0,27 %	0,00 %
	Ubuntu Core 24 64-bit	0,13 %	−0,01 %

Kilde: store.steampowered.com/hwsurvey — kombinerede platformsdata (Windows, Mac og Linux).

Sammenlagt sider Windows på 93,47% af hele Steams brugerbase. OSX (Apple) 2.01% og Linux på 4,52%.

Det vil sige at for at ramme flest muligt bør vi understøtte som minimum Windows 10 og Windows 11.

Lanceringskampagne

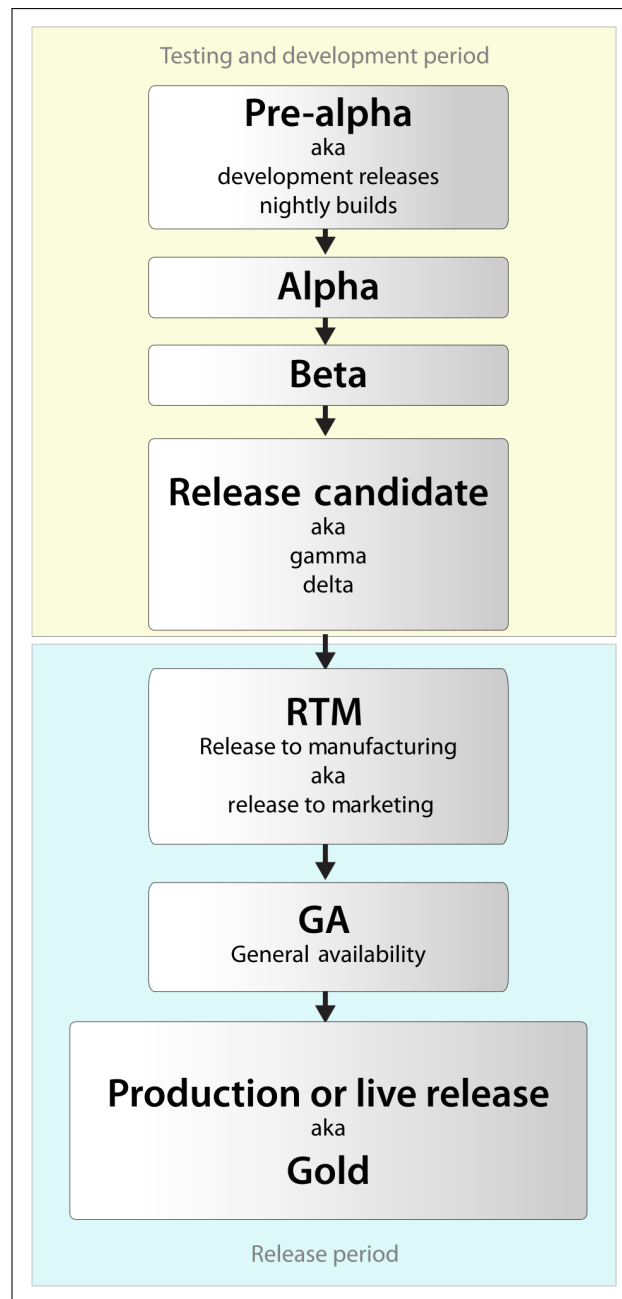
Fremtidige opdateringer

Early Access-development (versus “gammeldags” udvikling [golden edition])

11.3 Udgivningsstadie

Spilvirksomheder udgiver sjældent et spil 100% første gang. Man taler ofte om “The software release life cycle” — denne beskriver udviklings-, test- og distribueringsfaserne af et stykke software.¹²

¹²Wikipedia (apr. 2026b). *Software release life cycle*. Wikipedia. URL: https://en.wikipedia.org/wiki/Software_release_life_cycle.



Figur 8: Et eksempel af: “software release life cycle”Wikipedia, 2026b

I den traditionelle udviklingsmodel — ofte kaldet *golden edition*-modellen — udgives spillet først når det er færdigudviklet og gennemtestet. Spillet betragtes som låst (*gold*) når det er klar til distribution, og efterfølgende opdateringer er forbeholdt fejlrettelser. Denne model stiller høje krav til forudgående planlægning, da det er svært at ændre fundamentale designbeslutninger efter udgivelsen. Før i tiden, da man købte spil i fysiske udgaver på CD eller kassette, må det siges at have været ret svært at opdatere spillet efterfølgende for alle ens kunder, hvis man lige skulle rette en lille fejl. Med internettets udbredelse og digitale distributionsplatforme som Steam mistede golden edition-modellen sin praktiske nødvendighed — det blev pludselig trivielt at sende opdateringer ud til alle spillere på én gang, hvilket banede vejen for mere iterative udviklingsmodeller som Early Access.

En nyere, og i dag meget udbredt alternativ model er *Early Access*. Early Access er hvor spillet udgives i en ufærdig tilstand (ofte til en reduceret pris). Spillerne får dermed adgang til spillet tidligt i udviklingsforløbet og bidrager aktivt til dets forbedring gennem feedback og stress test af systemer. Denne model er særligt populær inden for uafhængigt producerede spil (indie-spil), hvor udviklerne ikke nødvendigvis har ressourcer til en fuldstændig færdigudviklet udgivelse. Velkendte eksempler på spil der benyttede Early Access er *Minecraft* og *Satisfactory*.

The software release cycle består af to hovedfaser delt op i 7 stadier. Den første fase er test- og udviklingsfasen, herunder er følgende stadier:

1. **Pre-alpha**

Pre-alpha er det første stadie et spil har. Dette stadie inkluderer alt før mere formel testing går i gang. Pre-alpha-versioner er ofte den version man har efter en dags udvikling.

2. **Alpha**

Alpha-stadiet betegner spillet når det stadig er i en intern testfase, og altså endnu ikke eksponeret for omverdenen. Alpha versioner har ofte mange ufuldkomne elementer, men stadig de centrale elementer begynder at tage form.

3. **Beta**

I beta-stadiet er spillet for det meste færdigt, det har ofte dog en helt masse ukendte/skjulte fejl som løbende bliver udbedret henimod den fulde version.

4. **Release candidate**

Release candidate (udgivelseskandidaten), er den beta-version med potentialet til at udgøre et stabilt produkt, altså et næsten fejlfrit produkt.

Herefter overgår produktet til udgivnings fasen (Release period)

1. **RTM**

RTM (*Release to Manufacturing*) er det stadie hvor produktet erklæres færdigt og sendes til produktion. Historisk set betød dette at spillet blev kopieret til fysiske medier som CD'er eller kassetter klar til distribution — deraf navnet.

2. **GA**

GA (*General Availability*) er det stadie hvor produktet bliver gjort tilgængeligt for offentligheden. Det er på dette tidspunkt, at spillet kan købes og spilles af alle.

3. **Production / Gold**

Gold-stadiet betegner den endelige, stabile version af produktet. Spillet betragtes som færdigt og låst — efterfølgende ændringer begrænses til kritiske fejlrettelser. Det er denne model der kendetegner den klassiske *golden edition*-udviklingsmodel.

Skulle **SPIL NAVN** udgives kommercielt, ville Early Access-modellen være oplagt for os, da spillet stadig har ufuldkomne features og ville kunne drage fordel af løbende brugerfeedback og økonomisk støtte i en tidlig fase.

Evt. regional prissættelse

Bibliografi

- Anderson, Carter (2022). *Bevy's First Birthday*. URL: <https://bevy.org/news/bevys-first-birthday/>.
- Cleanup, The Ocean (2026). *The Great Pacific Garbage Patch*. Dateret 23. april 2026. URL: <https://www.theoceancleanup.com/great-pacific-garbage-patch>.
- Frans Ørsted Andersen lektor, Ph.d. (2022). „Flow i undervisningen – hvorfor, hvornår og hvordan?“ I: URL: <https://emu.dk/grundskole/paedagogik-og-didaktik/undervisningsformer/flow-i-undervisningen-hvorfor-hvornaar-og>.
- National Geographic Society (apr. 2025). *Great Pacific Garbage Patch*. Dateret 20. april 2026. URL: <https://education.nationalgeographic.org/resource/great-pacific-garbage-patch/>.
- Ramsey, Sophie Esther (sep. 2023). *Anemoia & Gen Z's '80s dream*. The Varsity. URL: <https://thevarsity.ca/2023/09/10/anemoia-gen-zs-80s-dream/>.
- Steam, Valve (mar. 2026). *Steam User Survey 2026*. URL: <https://store.steampowered.com/hwsurvey/Steam-Hardware-Software-Survey-Welcome-to-Steam>.
- Wikipedia (2026a). *Flow (psychology)*. Wikipedia.com. URL: [https://en.wikipedia.org/wiki/Flow_\(psychology\)](https://en.wikipedia.org/wiki/Flow_(psychology)).
- (apr. 2026b). *Software release life cycle*. Wikipedia. URL: https://en.wikipedia.org/wiki/Software_release_life_cycle.

12 Bilag

.1 Appendiks A “brainstorm”

Introduktion Projektets indledende brainstorm, her produktideer til det sidenhen valgte oplæg. Ikke medtagede ideer er til de andre oplæg, som sidenhen blev kasseret.

Scan og medtag overstående

.2 Appendiks B “afleverede projektbeskrivelse”

Introduktion Projektets beskrivelse, godkendt og afleveret d. 16. marts 2026.

Projektbeskrivelse
eksamensprojekt DDU (teknikfag A)

Jeppe Bøgeskov og Alexander Schade

16. marts 2026

Valgte projektoplæg

Teknikfag A – Digitalt Design og Udvikling
Slagelse HTX - 2026

Projektoplæg 2

Tema

Opsamling af plastik

Formål

Udvikling af et læringsspil der omhandler indsamling af plastik og hvorfor det er et problem, hvis det ikke gøres, eks.: Resource mangel eller plastik øer i Stillehavet

Beskrivelse

Spillet skal skabe awareness om plastik oprydning og hvad det vil sige at være ansvarlig for hvad man efterlader, særligt mht. plastik. (det kunne også at stille spørgsmålstegn til hvorfor der ikke er en eneste plastik skraldespand på skolen – måske er man på quest efter plastik?).

Særlige krav (husk i bliver målt på disse til eksamen)

1. Brugeren skal igennem spillet lære at plastik skal indsamles og afleveres korrekt.
2. Spillet kan anvende simulering, eks. Af et økosystem der bliver fyldt med plastik.
3. Spillet skal have en modstander med grundlæggende egenskaber (eks. Patrol script)
4. Adskillige assets skal være lavet fra bunden og have udførlig begrundet sammenhæng med en valgt målgruppe.
5. Spillet skal gennemføre minimum 5 runder brugertests, der anvendes til at forbedre spillet.
6. Spillet skal som minimum have en funktionel og en æstetisk prototype.
7. Produktet skal deles eller afleveres i en interaktiv udgave.
8. Produktet skal optages som en "playthrough" film via screengrab eller med mobil.

Problemformulering

Udkastet til problemformuleringen, der findes interessant er:

Hvordan kan man lave et spil med havstrømme, der er interessant at navigere i, som har til formål at bringe et moralsk budskab om forurening og *vis* det til spilleren, ikke *fortælle* det, men heller ikke forcere det frem?

Tekniske problemer

1. Hvordan kan vi gøre, at båd-movement ikke er til gene for gameplay og forekommer naturligt?
2. Hvordan kan vi lave interessant gameplay, der får spilleren i flow, når der er tale om et top-down-spil med et fixed-camera?
3. Hvordan kan vi lave en overordnet progression, der giver spilleren lyst til at fortsætte og dygtiggøre sig?

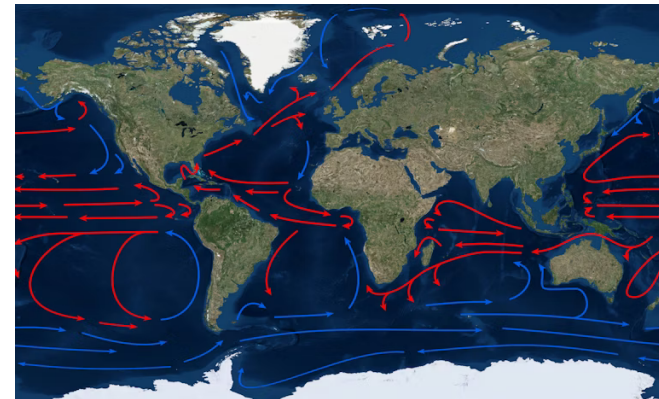
Udkast til problemanalyse

0.1 Havstrømme

Havstrømme opstår som følge af klimaforhold, herunder temperatur-, vind-, og saltforhold, se¹. Som det fremgår af Figur 1, eksisterer havstrømme på en meget stor skala, så de på en menneskelig skala vil fremstå som strømmende fra en retning til den modsatte. For at få en forståelse af strømmene, bliver de således nødt til at være mere tegneserieagtige.

0.2 Opløsninger

Det fremgår af Steam² Hardware og Software Surveys, at den mest anvendte skærmopløsning er 1920x1080 pixels, som 45,04% anvender, og den næstmest anvendte skærmopløsning er 2560x1440 pixels, som 38,64%, altså udgør de to skærmopløsninger 83,68% tilsammen.



Figur 1: Viser, hvordan havstrømmene ser ud.

¹Illustreret videnskab (u.d.). *Hvordan opstår havstrømme?* URL: <https://illvid.dk/naturen/hvordan-opstaar-havstroemme>.

²Steam er en af de mest populære game launchers og udgiver månedligt en oversigt over deres brugeres maskinelle setups, hvilket er utrolig brugbart for en spiludvikler, så man kan målrette sit spil

Overvejelser om projektets indhold

Projektet skal være top-down med en player avatar bestående af en båd-sprite. Desuden skal der være affaldsstykker, der påvirkes af havstrøm og ophober sig, hvorefter de bliver til fjender. Spillerens avatar skal også påvirkes af havstrømmene og må således korrigere for at bevare sin position. Spilleren bekæmper monstre og indsamler plastikstykkerne som valuta, og kan således komme videre. I intermissioner kan spilleren købe items og opgraderes.

Vi kan tilføje opgraderinger og forskellige fjender og krav til, hvordan de opstår, når vi har lavet den primære spil sløjfe. Ligeså kan vi kigge på en fortælling, når vi har gjort det, mens world-building er en prioritet, og budskabet skal formidles uafhængigt af fortællingen.

Tidsplan

Tid-resurser

90 moduler a en times varighed.

Vigtige datoer

Projektstart mandag d. 9. marts 2026

Frist for godkendelse af projektbeskrivelse mandag d. 16. marts 2026

Midtvejsdato mandag d. 13. april 2026

Projektugestart fredag d. 23. april 2026

Projektugeslut fredag d. 29. april 2026

Afleveringsfrist for produkt og rapport torsdag d. 18. maj 2026

Arbejdsuger (sprints)

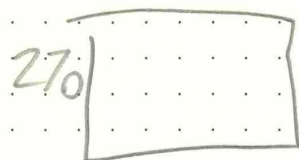
Ugenr.	startdag
11	mandag d. 9. marts 2026
12	mandag d. 16. marts 2026
13	mandag d. 23. marts 2026
14	mandag d. 30. marts 2026
15	mandag d. 6. april 2026
16	mandag d. 13. april 2026 (<i>Obs. projektuge</i>)
17	mandag d. 20. april 2026
18	mandag d. 27. april 2026
19	mandag d. 4. maj 2026
20	mandag d. 18. maj 2026 (<i>Obs. afleveringsfrist</i>)

Tabel 1: Viser de pågældende uger, hvor projektet er verserende.

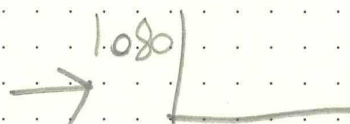
Tidsplanen er styret via GitHubs project-managamentet med udgangspunkt i SCRUM-metoden. Her ses det som et roadmap: <https://github.com/orgs/ZBC-Slagelse-HTX-X/projects/6/views/4>

.3 Appendiks C “opløsningovervejelser”

Introduktion Tegning, der viser, hvordan vi sikrer pixel-perfect skalering på 1920x1080 pixel ved først at render et vindue på 270x480 pixel og sidenhen skalere det med en faktor fire. Omvendt ses det, at når billedforholdet på 10x16 håndteres ved sorte barer.



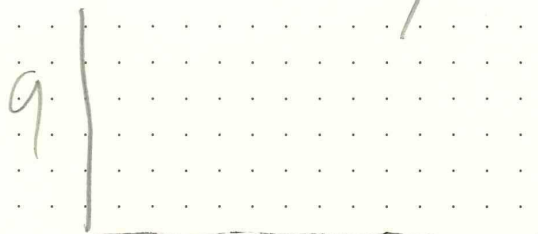
480



1080

1920

4



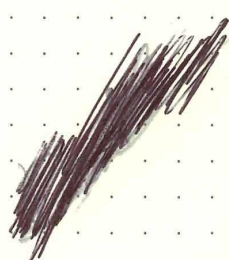
16

0,5€

10

0,5€

16



.4 Appendiks D “stand-up-referater”

Introduktion Journal over referaterne fra projektets stand-ups.

Dato	Referater
d. 23. marts	• Alexander kigger på logic-grid, da optimering af bevægelse afhænger af det. • Fortsættelse på movement (Alexander). • Jeppe påbegynder pixel-art, først moodboard, sidenhen map og sprites. • Evt. afprøvning af forskellige movement-varianter (rotation og stick-velegnethed) • Jeppe tager tidlig ferie fra på onsdag, altså er han her ikke i de allokerede moduler på torsdag d. 26.
d. 26. marts	• Alexander har egenhændigt implementeret splash screens og game states
d. 14. april	• Jeppe har lavet pixel-art, herunder tundåse og plastikflaske • Alexander har kigget på hit detection og gjort, så sigtekornets distance til spilleren har et minimum.
d. 16. april	• Alexander har skrevet på rapport. • Jeppe har lavet pixel-art, en tom vanddunk. • Vi har d.d. aftalt, at vi i projektugen vil fokusere meget energi på at skrive projektrapporten.

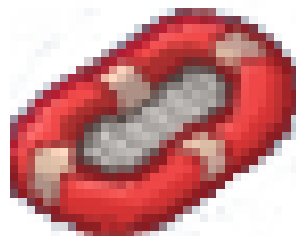
.5 Appendiks E “Sprites”

Spritesne er alle markant opskaleret og kan derfor fremgå lidt “sjove” på billederne.

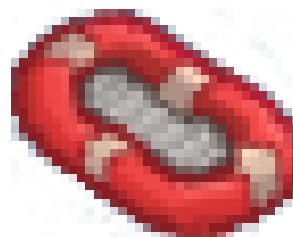
Vest



Syd-vest



Syd-øst



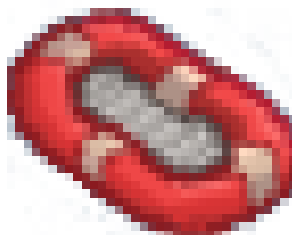
Syd



Øst



Nord-vest



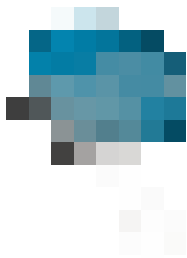
Nord-øst



Nord



Vest



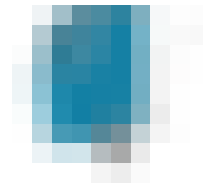
Syd-vest



Syd-øst



Syd



Øst



Nord-vest



Nord-øst



Nord



Højre front



Højre ryg

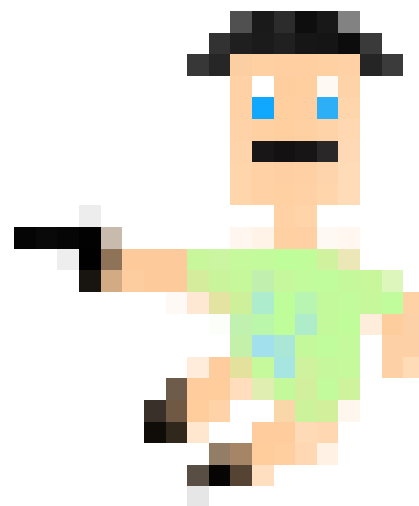
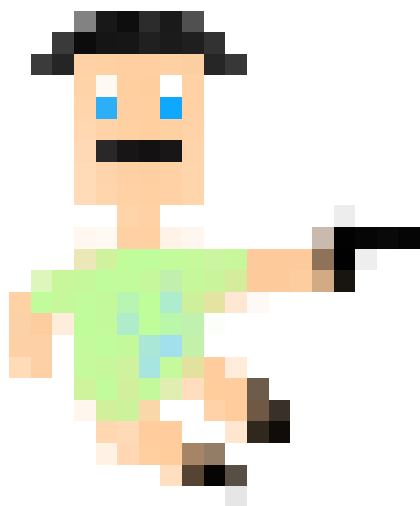


Venstre ryg

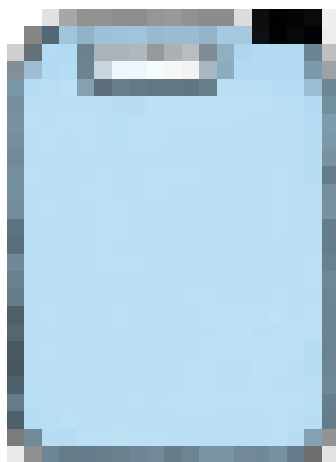


Venstre front

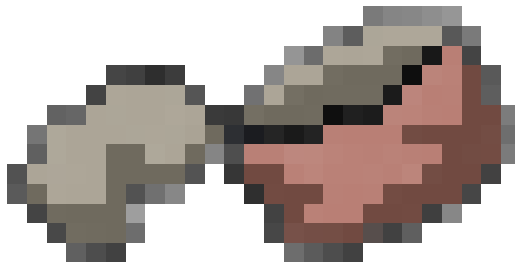




Vanddunk



Dåse



Vandflaske



.6 Appendiks F “Moodboard”

